

Écriture de votre première application Django, 4ème partie 📄

Ce tutoriel commence là où le [tutoriel 3](#) s'achève. Nous continuons l'application de sondage Web et allons nous focaliser sur la gestion de formulaire et sur la réduction du code.



Où obtenir de l'aide :

Si vous rencontrez des problèmes dans le parcours de ce tutoriel, rendez-vous dans la section [Obtenir de l'aide de la FAQ](#).

Écriture d'un formulaire minimal 📄

Nous allons mettre à jour le gabarit de la page de détail (« `polls/details.html` ») du tutoriel précédent, de manière à ce que le gabarit contienne une balise HTML `<form>` :

`polls/templates/polls/detail.html` 📄

```
<form action="{% url 'polls:vote' question.id %}" method="post">
{% csrf_token %}
<fieldset>
  <legend><h1>{{ question.question_text }}</h1></legend>
  {% if error_message %}<p><strong>{{ error_message }}</strong></p>{% endif %}
  {% for choice in question.choice_set.all %}
    <input type="radio" name="choice" id="choice{% forloop.counter %}" value="{{ choice.id }}">
    <label for="choice{% forloop.counter %}">{{ choice.choice_text }}</label><br>
  {% endfor %}
</fieldset>
<input type="submit" value="Vote">
</form>
```

Un résumé rapide :

- Ce gabarit affiche un bouton radio pour chaque choix de question. L'attribut **value** de chaque bouton radio correspond à l'ID du vote choisi. Le nom (**name**) de chaque bouton radio est **"choice"**. Cela signifie que lorsque quelqu'un sélectionne l'un des boutons radio et valide le formulaire, les données POST **choice=#** (où # est l'identifiant du choix sélectionné) seront envoyées. Ce sont les concepts de base des formulaires HTML.
- Nous avons défini `{% url 'polls:vote' question.id %}` comme attribut **action** du formulaire, et nous avons précisé **method="post"**. L'utilisation de **method="post"** (par opposition à **method="get"**) est très importante, puisque le fait de valider ce formulaire va entraîner des modifications de données sur le serveur. À chaque fois qu'un formulaire modifie des données sur le serveur, vous devez utiliser **method="post"**. Cela ne concerne pas uniquement Django ; c'est une bonne pratique à adopter en tant que développeur Web.
- **forloop.counter** indique combien de fois la balise `for` a exécuté sa boucle.
- Comme nous créons un formulaire POST (qui modifie potentiellement des données), il faut se préoccuper des attaques inter-sites. Heureusement, vous ne devez pas réfléchir trop longtemps car Django offre un moyen pratique à utiliser pour s'en protéger. En bref, tous les formulaires POST destinés à des URL internes doivent utiliser la balise de gabarit `{% csrf_token %}`.

Maintenant, nous allons créer une vue Django qui récupère les données envoyées pour nous permettre de les exploiter. Souvenez-vous, dans le [tutoriel 3](#), nous avons créé un `URLconf` pour l'application de sondage contenant cette ligne :

`polls/urls.py` 📄

```
path("<int:question_id>/vote/", views.vote, name="vote"),
```

Getting Help

Nous avons également créé une implémentation rudimentaire de la fonction **vote()**. Créons maintenant une version fonctionnelle. Ajoutez ce qui suit dans le fichier `polls/views.py` :

`polls/views.py` 📄

Langue : fr

Version de la documentation : 6.0



```

from django.db.models import F
from django.http import HttpResponseRedirect
from django.shortcuts import get_object_or_404, render
from django.urls import reverse

from .models import Choice, Question

# ...
def vote(request, question_id):
    question = get_object_or_404(Question, pk=question_id)
    try:
        selected_choice = question.choice_set.get(pk=request.POST["choice"])
    except (KeyError, Choice.DoesNotExist):
        # Redisplay the question voting form.
        return render(
            request,
            "polls/detail.html",
            {
                "question": question,
                "error_message": "You didn't select a choice.",
            },
        )
    else:
        selected_choice.votes = F("votes") + 1
        selected_choice.save()
        # Always return an HttpResponseRedirect after successfully dealing
        # with POST data. This prevents data from being posted twice if a
        # user hits the Back button.
        return HttpResponseRedirect(reverse("polls:results", args=(question.id,)))

```

Ce code contient quelques points encore non abordés dans ce tutoriel :

- **request.POST** est un objet similaire à un dictionnaire qui vous permet d'accéder aux données envoyées par leurs clés. Dans ce cas, **request.POST['choice']** renvoie l'ID du choix sélectionné, sous forme d'une chaîne de caractères. Les valeurs dans **request.POST** sont toujours des chaînes de caractères.
- Notez que Django dispose aussi de **request.GET** pour accéder aux données GET de la même manière – mais nous utilisons explicitement **request.POST** dans notre code, pour s'assurer que les données ne sont modifiées que par des requêtes POST.
- **request.POST['choice']** lèvera une exception **KeyError** si **choice** n'est pas spécifié dans les données POST. Le code ci-dessus vérifie qu'une exception **KeyError** n'est pas levée et réaffiche le formulaire de question avec un message d'erreur si **choice** n'est pas rempli.
- **F("votes") + 1** demande à la base de données d'augmenter le nombre de votes de 1.
- Après l'incrémentation du nombre de votes du choix, le code renvoie une **HttpResponseRedirect** plutôt qu'une **HttpResponse** normale. **HttpResponseRedirect** prend un seul paramètre : l'URL vers laquelle l'utilisateur va être redirigé (voir le point suivant pour la manière de construire cette URL dans ce cas).
- Comme le commentaire Python l'indique, vous devez systématiquement renvoyer une **HttpResponseRedirect** après avoir correctement traité les données POST. Ceci n'est pas valable uniquement avec Django, c'est une bonne pratique du développement Web.
- Dans cet exemple, nous utilisons la fonction **reverse()** dans le constructeur de **HttpResponseRedirect**. Cette fonction nous évite de coder en dur une URL dans une vue. On lui donne en paramètre la vue vers laquelle nous voulons rediriger ainsi que la partie variable de l'URL qui pointe vers cette vue. Dans ce cas, en utilisant l'URLconf défini dans la [partie 3](#) de ce tutoriel, l'appel de la fonction **reverse()** va renvoyer la chaîne de caractères :

```
"/polls/3/results/"
```

où **3** est la valeur de **question.id**. Cette URL de redirection va ensuite appeler la vue '**results**' pour afficher la page finale.

Comme expliqué dans la [partie 3](#) de ce tutoriel, **request** est un objet **HttpRequest**. Pour plus d'informations sur les objets **HttpRequest**, voir la [documentation des requêtes et réponses](#).

Après le vote d'une personne dans une question, la vue **vote()** redirige vers la page de résultats de la question. Écrivons cette vue :

polls/views.py ¶

Getting Help 📖

```

from django.shortcuts import get_object_or_404, render

def results(request, question_id):
    question = get_object_or_404(Question, pk=question_id)
    return render(request, "polls/results.html", {"question": question})

```



C'est presque exactement la même que la vue `detail()` du [tutoriel 3](#). La seule différence est le nom du gabarit. Nous éliminerons cette redondance plus tard.

Écrivons maintenant le gabarit `polls/results.html` :

polls/templates/polls/results.html 1

```
<h1>{{ question.question_text }}</h1>

<ul>
{% for choice in question.choice_set.all %}
  <li>{{ choice.choice_text }} -- {{ choice.votes }} vote{{ choice.votes|pluralize }}</li>
{% endfor %}
</ul>

<a href="{% url 'polls:detail' question.id %}">Vote again?</a>
```

Maintenant, rendez-vous à la page `/polls/1/` avec votre navigateur et votez pour la question proposée. Vous devriez voir une page de résultats qui sera mise à jour à chaque fois que vous voterez. Si vous validez le formulaire sans avoir coché votre choix, vous devriez voir le message d'erreur.

Utilisation des vues génériques : moins de code, c'est mieux 1

Les vues `detail()` (cf. [tutoriel 3](#)) et `results()` sont très courtes – et comme mentionné précédemment, redondantes. La vue `index()` qui affiche une liste de sondages est similaire.

Ces vues représentent un cas classique du développement Web : récupérer les données depuis la base de données suivant un paramètre contenu dans l'URL, charger un gabarit et renvoyer le gabarit interprété. Ce cas est tellement classique que Django propose un raccourci, appelé le système de « vues génériques ».

Les vues génériques ajoutent une couche d'abstraction des procédés courants au point où vous n'avez même plus besoin d'écrire du code Python pour écrire une application. Par exemple, les vues génériques `ListView` et `DetailView` implémentent respectivement les concepts de « afficher une liste d'objets » et « afficher une page détaillée pour un type particulier d'objet ».

Nous allons convertir notre application de sondage pour qu'elle utilise le système de vues génériques. Nous pourrions ainsi supprimer une partie de notre code. Nous avons quelques pas à faire pour effectuer cette conversion. Nous allons :

1. Convertir l'URLconf.
2. Supprimer quelques anciennes vues désormais inutiles.
3. Introduire de nouvelles vues basées sur les vues génériques de Django.

Lisez la suite pour plus de détails.



Pourquoi ces changements de code ?

En général, lorsque vous écrivez une application Django, vous devez estimer si les vues génériques correspondent bien à vos besoins et, le cas échéant, vous les utiliserez dès le début, plutôt que de réarranger votre code à mi-chemin. Mais ce tutoriel s'est concentré intentionnellement sur l'écriture des vues « à la dure » jusqu'à ce point, pour mettre l'accent sur les concepts de base.

Tout comme vous devez posséder des bases de maths avant de commencer à utiliser une calculatrice.

Correction de l'URLconf 1

Tout d'abord, ouvrez la configuration d'URL `polls/urls.py` et modifiez-la ainsi :

polls/urls.py 1

```
from django.urls import path

from . import views

app_name = "polls"
urlpatterns = [
    path("", views.IndexView.as_view(), name="index"),
    path("<int:pk>/", views.DetailView.as_view(), name="detail"),
    path("<int:pk>/results/", views.ResultsView.as_view(), name="results"),
    path("<int:question_id>/vote/", views.vote, name="vote"),
]
```

Getting Help

Notez que le nom des motifs de correspondance dans les chaînes de chemin des deuxième et troisième motifs ont changé de `<question_id>` à `<pk>`. Ceci est nécessaire car la vue générique `DetailView` sera utilisée pour remplacer les vues `detail()` et `results()`, et que cette vue s'attend à ce que la valeur de clé primaire capturée dans l'URL soit nommée `"pk"`.



Correction des vues 1

Ensuite, nous allons enlever les anciennes vues **index**, **detail** et **results** et utiliser à la place des vues génériques de Django. Pour cela, ouvrez le fichier **polls/views.py** et modifiez-le de cette façon :

```
polls/views.py 1

from django.db.models import F
from django.http import HttpResponseRedirect
from django.shortcuts import get_object_or_404, render
from django.urls import reverse
from django.views import generic

from .models import Choice, Question

class IndexView(generic.ListView):
    template_name = "polls/index.html"
    context_object_name = "latest_question_list"

    def get_queryset(self):
        """Return the last five published questions."""
        return Question.objects.order_by("-pub_date")[:5]

class DetailView(generic.DetailView):
    model = Question
    template_name = "polls/detail.html"

class ResultsView(generic.DetailView):
    model = Question
    template_name = "polls/results.html"

def vote(request, question_id):
    # same as above, no changes needed.
    ...
```

Chaque vue générique doit connaître le modèle sur lequel elle agira. Pour cela, on utilise soit l'attribut **model** (dans cet exemple, **model = Question** pour **DetailView** et **ResultsView**), soit on définit la méthode **get_queryset()** (tel qu'illustré pour la vue **IndexView**).

Par défaut, la vue générique **DetailView** utilise un gabarit appelé **<nom app>/<nom modèle>_detail.html**. Dans notre cas, elle utiliserait le gabarit **"polls/question_detail.html"**. L'attribut **template_name** est utilisé pour signifier à Django d'utiliser un nom de gabarit spécifique plutôt que le nom de gabarit par défaut. Nous avons aussi indiqué le paramètre **template_name** pour la vue de liste **results**, ce qui permet de différencier l'apparence du rendu des vues « results » et « detail », même s'il s'agit dans les deux cas de vues **DetailView** à la base.

De la même façon, la vue générique **ListView** utilise par défaut un gabarit appelé **<nom app>/<nom modèle>_list.html** ; nous utilisons **template_name** pour indiquer à **ListView** d'utiliser notre gabarit existant **"polls/index.html"**.

Dans les parties précédentes de ce tutoriel, les templates ont été renseignés avec un contexte qui contenait les variables de contexte **question** et **latest_question_list**. Pour **DetailView**, la variable **question** est fournie automatiquement ; comme nous utilisons un modèle nommé **Question**, Django sait donner un nom approprié à la variable de contexte. Cependant, pour **ListView**, la variable de contexte générée automatiquement s'appelle **question_list**. Pour changer cela, nous fournissons l'attribut **context_object_name** pour indiquer que nous souhaitons plutôt la nommer **latest_question_list**. Il serait aussi possible de modifier les templates en utilisant les nouveaux nom de variables par défaut, mais il est beaucoup plus simple d'indiquer à Django les noms de variables que nous souhaitons.

Lancez le serveur et utilisez votre nouvelle application de sondage basée sur les vues génériques.

Pour plus de détails sur les vues génériques, voir la documentation des vues génériques.

Lorsque vous êtes à l'aise avec les formulaires et les vues génériques, lisez la 5ème partie de ce tutoriel pour apprendre comment tester notre application de sondage.

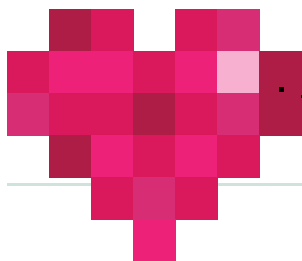
Support Django!

Sommaire

Getting Help

- Écriture de votre première application Django, 4ème partie
 - Écriture d'un formulaire minimal
 - Utilisation des vues génériques : moins de code, c'est mieux
 - Correction de l'URLconf





- [Correction des vues](#)

Obtenir de l'aide

FAQ

Essayez la FAQ, vous y trouverez des réponses à de nombreuses questions courantes.

Index, Index des modules, or Sommaire

Pratique lorsqu'on cherche des informations précises.

Django Discord Server

Join the Django Discord Community.

Official Django Forum

Join the community on the Django Forum.

Ticket tracker

Signalez des bogues de Django ou de sa documentation dans notre système de suivi de tickets.

Télécharger :

Hors ligne (Django 6.0) : [HTML](#) | [PDF](#) | [ePub](#)

Offert par [Read the Docs](#).

Diamond and Platinum Members

Sentry

Monitor your Django Code Resolve performance bottlenecks and errors using monitoring, replays, logs and Seer an AI agent for debugging.



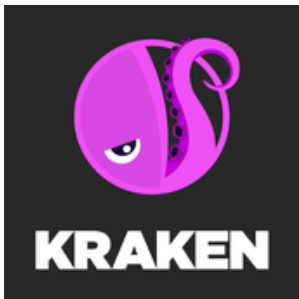
JetBrains

JetBrains delivers intelligent software solutions that make developers more productive by simplifying their challenging tasks, automating the routine, and helping them adopt the best development practices. PyCharm is the Python IDE for Professional Developers by JetBrains providing a complete set of tools for productive Python, Web and scientific development.



Getting Help





Kraken Tech

Kraken is the most-loved operating system for energy. Powered by our Utility-Grade AI™ and deep industry know-how, we help utilities transform their technology and operations so they can lead the energy transition. Delivering better outcomes from generation through distribution to supply, Kraken powers 70+ million accounts worldwide, and is on a mission to make a big, green dent in the universe.



PostHog

PostHog is an all-in-one developer platform for building successful products. We provide product analytics, web analytics, session replay, error tracking, feature flags, experiments, surveys, LLM observability, data warehouse, CDP, workflows, logs, and an AI product assistant to help debug your code, ship features faster, and keep all your usage and customer data in one stack.

Learn More

- About Django
- Getting Started with Django
- Team Organization
- Django Software Foundation
- Code of Conduct
- Diversity Statement

Get Involved

- Join a Group
- Contribute to Django
- Submit a Bug
- Report a Security Issue
- Individual membership

Get Help

- Getting Help FAQ
- Django Discord
- Official Django Forum

Getting Help



Follow Us

- GitHub
- X
- Fediverse (Mastodon)
- Bluesky
- LinkedIn
- News RSS

Support Us

- Sponsor Django
- Corporate membership
- Official merchandise store
- Benevity Workplace Giving Program

Getting Help

